

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC2006 – Software Engineering

Lab 4 Deliverables -

Supplementary Report: AI-Based Reconstruction & Feature Prototyping

Lab Group SCEF

Team 4

Team: Ian Tan, Robin Teoh, Nicholas Ng, Edwin Tan Qi Lun, Benjamin Ng

AI used: Claude Opus 5.5 (Claude Code Agent, VS Code)

1. Overview

An agentic AI tool was used to facilitate two different tasks: i) the re-generation of a deleted control class, and ii) the implementation of a new feature. These were carried out as part of a supplementary experiment exploring the use of AI as per Requirement 3.3.3 and 3.3.4 of the SC2006 Lab 4 Manual. As such, the subsequent code changes were contained within the `lab4-ai-regen` and `lab4-ai-feature` branches respectively, and were neither merged or staged into the URECAMatch production branch. As with previous AI reports, the team opted to use the Claude Opus 5.5 model by Anthropic.

2. Re-generation of Control Class

The goal of this section is to evaluate the performance of the chosen agentic AI tool in regenerating a deleted control class via inference from test cases and documentation.

2.1 Regeneration of Control Class: Methodology

In this experiment, the group chose to delete ApplicationController, implemented via `api/app/routers/applications.py`. A prompt detailing the deleted control class, its functionality, and what should regenerated was given to the AI model along with the following reference documentation:

a. Architecture Diagram
b. Class Diagram
c. A pared-down version of the SRS containing various key sections such as the FRs (See: SRS Section 4), NFRs (See: SRS Section 5), Use Case Descriptions (See: SRS Appendix)
d. `.txt` file containing filepaths to test-cases designed for the ApplicationController as per Requirement 3.2.4 of the Lab 4 manual

```
Restore a deleted control class, ApplicationController
(api/app/routers/applications.py) from its tests and design. You may make use of
the attached files for its specifications and documentation, but not past commit
history.

ADDITIONAL CONTEXT
URECAMatch is a FastAPI + Next.js app made to serve as an application web portal
for NTU's Undergraduate Research Programme on Campus (URECA). There are 3 Users:
Admins, who administrate the application platform and have administrative
privileges, Students, who are the prospective applicants to Projects, and
Supervisors, who supervise and accept Student applicants into their projects.

CODE TO REFERENCE (in additional to the relevant test cases file-path in the .txt
file)
```

```
- api/app/models/__init__.py: Application, ApplicationCreate, ApplicationStatus,
PreferenceRankingUpdate, Ok, ListingStatus, MatchStatus
- api/app/deps.py (StudentUser, StoreDep, SettingsDep) and api/app/errors.py
- api/app/services/windows.py (application window, current phase) and
api/app/services/placement.py (placed students, open offers)
- api/app/routers/proposals.py: a similar student flow, for style
- shared/openapi.json: the current contract.
- web/app/(student)/apply/[id]/page.tsx and
web/app/(student)/my-applications/page.tsx: how the screens use the API
```

Note that a certain portion of text in the prompt above is highlighted in blue font: this was an additional clause that was only added in the second attempt at regeneration.

2.2 Regeneration of Control Class: Results

The first attempt at re-generation yielded an interesting result: the agentic AI tool simply replicated the old file in its entirety. The following command was run to verify this.

```
diff -u <(git show HEAD:ureca-match-app/api/app/routers/applications.py)
"$(git rev-parse
--show-toplevel)/ureca-match-app/api/app/routers/applications.py"
```

This yielded an empty output, hence confirming that there were no differences between the original file and the newly regenerated file. Follow-up prompts were then made to the AI model to deduce the reasoning for this behaviour, and subsequently documented in Section 2.3 (Discussion) of this report.

The applications.py controller was then deleted again, and a second attempt was then made to prompt the agentic AI tool for regeneration. However, for this second attempt, an additional clause (highlighted in blue below) was added to the prompt:

```
Restore a deleted control class, ApplicationController
(api/app/routers/applications.py) from its tests and design. You may make
use of the attached files for its specifications and documentation, but not
past commit history.
```

The end result was a newly regenerated applications.py. The regenerated file and its difference is too verbose to display in this report, but is viewable on the `lab4-ai-regen` branch of the GitHub remote repository.

2.3 Regeneration of Control Class: Discussion / Reflection

Upon further prompting and investigation into the first behaviour exhibited by the agentic AI tool (where it generated an exact copy of the deleted file), it was revealed that the AI model had bypassed the instructions in the generation prompt and instead restored the file by accessing the current working commit tree, applying git diff, and subsequently regenerating an exact copy of the file.

There may be several possible reasons for this. The first may be loosely linked to a term called "evaluation awareness" in AI literature [1], where AI models would frequently "cheat" to achieve a task by observing a leaked answer (in this case, git history). This would lead to "contamination" of the generation results, which in this case meant the model simply duplicated the file from history. The second reason may simply be due to the ambiguous phrasing of the prompt wording, ie. "Restore a deleted control class, ApplicationController…", which led the AI to interpret the instruction in its most literal form, which was to "undo" and restore the deleted file back exactly as it were.

The second attempt with the added guard clause for constraints on regeneration yielded a more successful generation result. Functionally, the new Application Controller generated by the model behaved in the same way, with the exception of a single bug: it failed to coordinate API input restrictions with the UI sanitization checks.

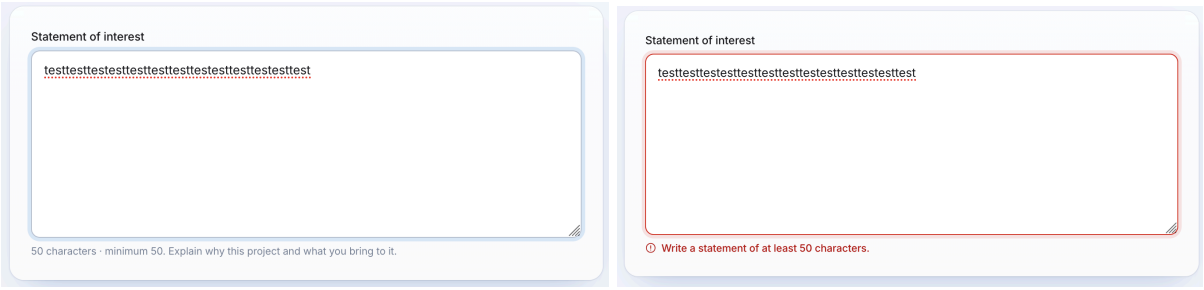


Fig. 1 Left, Input validation by UI element confirms Student wrote 50 characters. Right, form rejected by API after clicking the "Submit" button due to not meeting the 50-character length.

As can be seen in Fig. 1, when a Student is filling in their Statement of Interest, the UI form element performs its own validation to check if Student has filled in a minimum of 50 characters. However, the API generated by the new Application Controller class validates this differently, instead trimming whitespaces from its character count, hence resulting in a mismatch between the requirements between the screen input and the API. While the agentic AI tool excelled at regenerating the exact local behaviour of the Application Controller, it nevertheless falls short in accounting for systems-wide coherency.

3. AI-generated Feature

The goal of this section is to evaluate the performance of the chosen agentic AI tool in implementing an additional feature to URECAMatch. The chosen feature will be the addition of a chat system for communication between Student and Supervisor users.

3.1 AI-generated Feature: Methodology

In this experiment, the team chose to implement a chat system via the agentic AI tool, without any manual corrections or direct human involvement in editing the code. Several new requirement specifications were drafted to aid the AI model in generating this feature, namely:

- REQ-CHAT-1 A student and a supervisor can message each other only if the student has applied to one of the supervisor's projects or sent that supervisor a proposal (any status). Both accounts must be active. Admins have no access.
- REQ-CHAT-2 One conversation per student and supervisor pair. Either side can start it. Each side sees the other person's name and the project titles that link them.
- REQ-CHAT-3 A message is delivered in plain text, 1 to 2000 characters after trimming. A
- REQ-CHAT-4 Only the two participants as defined in REQ-CHAR-2 can read or post in a conversation.
- REQ-CHAT-5 Opening a conversation marks it read. Show unread counts per conversation, and a Messages item with an unread badge in the sidebar, like Notifications.
- REQ-CHAT-6 The recipient gets a notification through services/notify.py, linking to the conversation. No further notification for that conversation until they have read it.

This prompt was then given to the model on a fresh branch (lab4-ai-added-feature), with free reign to edit the code as necessary for implementation. The prompt can be seen below:

```
Build a new feature: chat between a student and a supervisor. The feature is not in
the SRS. However, an existing copy of the SRS, Class Diagram, Architecture Diagram,
and Use Cases is attached. Note that these only describe the current state of the
implementation, ie. this feature has yet not been defined in the documentation.
Instead, you may refer to the new requirements as defined below.


ADDITIONAL CONTEXT
URECAMatch is a FastAPI + Next.js app made to serve as an application web portal
for NTU's Undergraduate Research Programme on Campus (URECA). There are 3 Users:
Admins, who administrate the application platform and have administrative
privileges, Students, who are the prospective applicants to Projects, and
Supervisors, who supervise and accept Student applicants into their projects.
```

```
REQUIREMENTS
REQ-CHAT-1 A student and a supervisor can message each other only if the student
has applied to one of the supervisor's projects or sent that supervisor a proposal
(any status). Both accounts must be active. Admins have no access.
REQ-CHAT-2 One conversation per student and supervisor pair. Either side can start
it. Each side sees the other person's name and the project titles that link them.
REQ-CHAT-3 A message is delivered in plain text, 1 to 2000 characters after
trimming. A
REQ-CHAT-4 Only the two participants as defined in REQ-CHAR-2 can read or post in a
conversation.
REQ-CHAT-5 Opening a conversation marks it read. Show unread counts per
conversation, and a Messages item with an unread badge in the sidebar, like
Notifications.
REQ-CHAT-6 The recipient gets a notification through services/notify.py, linking to
the conversation. No further notification for that conversation until they have
read it.
```

3.2 AI-generated Feature: Results

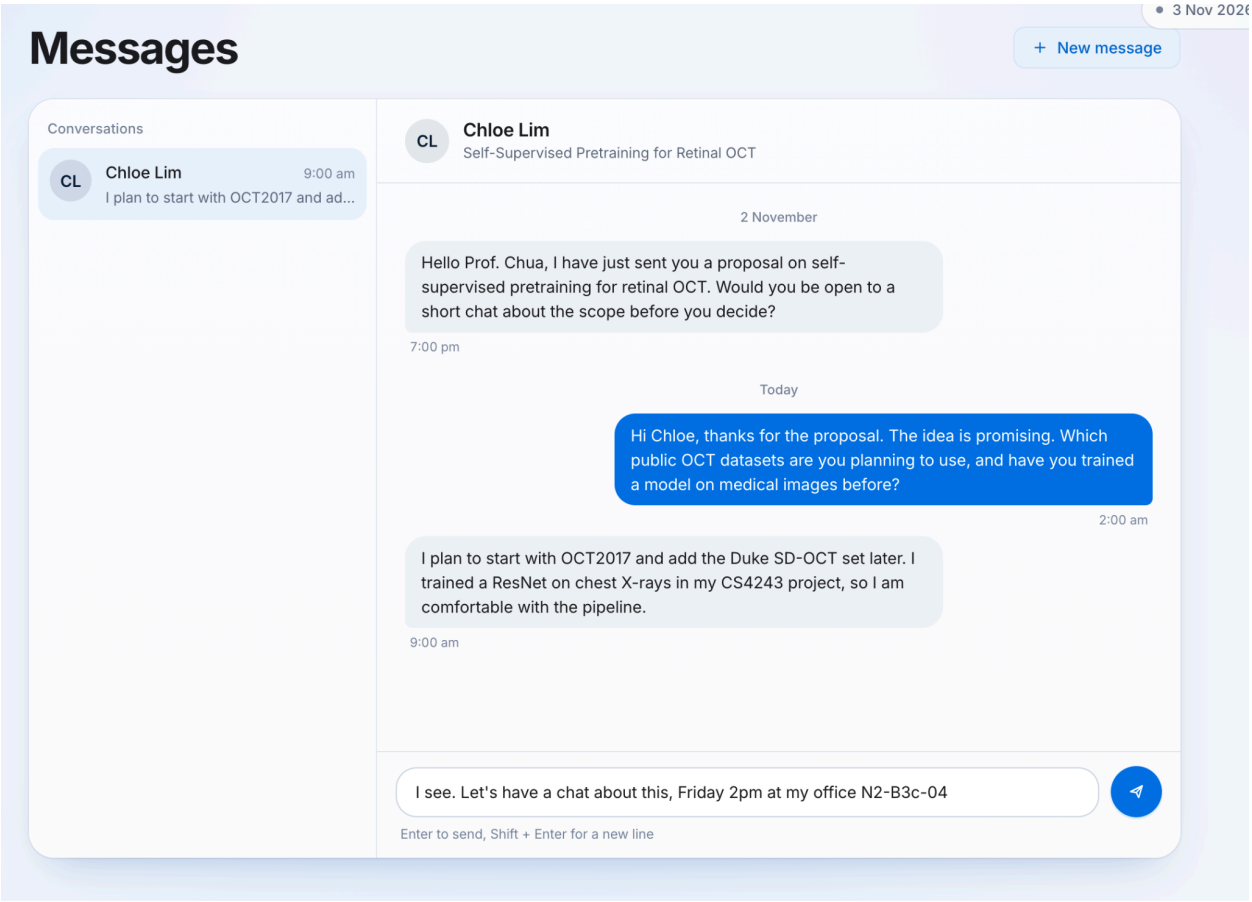


Fig 2. Screenshot of the AI-implemented chat feature in the URECAMatch app.

The agentic AI tool completed the feature in a single session without any assistance from the developer team. Running a `git diff` command showed that 30 existing files were modified, with 7 new files added to support the new feature. The chat feature was added as a separate screen, accessible through the sidebar as seen in Fig. 2. The modifications are summarized in Table 1 below.

Layer	Modifications made by the AI agentic tool
Database	- New database table entries for conversations and messages - A new type of notification: `message_received`
Control	- A new ChatController (`api/app/routers/chat.py`) and chat service (`api/app/services/chat.py`). - Seven new data-store methods, implemented for both mock and Supabase stores to save chats to persistent memory - An unread_messages count added to GET /me end-point - A new e-mail template corresponding to the newly added “message_received” notification type.
Front-end	- A new screen displaying messages as a sidebar with the list of conversations, and the main windowing displaying the selected conversation. This screen is accessible through the existing sidebar in URECAMatch. - A "New message" button that allows for the sender to pick a eligible recipient (eg. a Student can choose a Supervisor to message). This starts a new conversation in the list view.
Tests	- A new test file, `testREQCHAT.py`, with 29 new tests.

Table 1. Summary of changes made to URECAMatch by the AI agentic tool to implement the chat feature.

3.3 AI-generated Feature: Discussion

In general, there were no major implementation or correctness errors made by the AI agentic tool in the process of implementing the chat feature. All 6 defined REQs (See: Section 3.1 of this Supplementary Report) were fulfilled, with the final build passing all previously existing tests as well as the newly added tests for the chat feature.

Interestingly, the AI model also independently added safeguards in the debug mode of URECAMatch: since the clock is frozen in certain debug settings, sent messages would all contain the same timestamp, making their send order ambiguous. Without prompting, the agentic tool added a microsecond between each consecutively sent timestamp to combat this. The AI model also made other modifications to the repository documentation to document this new feature, even though no such request was made in the original prompt. Last but not least, the development team found that a new email template was added for notifications despite not being specified in the provided requirements.

While the team was initially concerned that the agentic AI tool would be unable to adequately engineer a feature that would require extensive modifications to multiple layers in the URECAMatch app architecture, it was found that the chat feature was implemented relatively well. We attributed this to the fact that the requirements for the new feature were well-defined by the 6 rules we gave in the prompt, which hence provided adequate constraints for the AI model to work with. However, as the model AI tended to add unrequested features, a decision was made to not include this feature in the final product, as not enough time was left in the development cycle for the team to independently verify all the changes that were made, hence increasing the risk of regression bugs.

4. Conclusion

In this supplementary report, two experiments were carried out using an AI agentic tool running on Claude Opus 5.5: i) the re-generation of a deleted control class, and ii) the implementation of a new feature. Although the agentic tool made no major (observed) correctness errors in both experiments, the team chose not to accept the modifications made by the AI into the final production code. This was due to the agentic AI tool having made multiple changes outside of the requested scope as elaborated above (See: Section 3.3), as well as the lack of time for a proper human review of its edited code.

5. Bibliography

[1] Anthropic, “Eval awareness in Claude Opus 4.6’s BrowseComp performance,” Oct. 18, 2026. https://www.anthropic.com/research/eval-awareness-browsecomp